Конспект лекции 6: Архитектура Transformer

Липкин С.М.

1 От RNN к Transformer

1.1 Проблемы RNN

- Последовательная обработка: $O(n)$ шагов, нет параллелизма на GPU.
- Затухающие градиенты: связи на расстоянии $>$ 50–100 токенов теряются.
- LSTM/GRU помогли, но не решили проблему кардинально.
- Механизм Attention (Bahdanau, 2015) — шаг вперёд, но всё ещё на базе RNN.

1.2 Решение: Transformer (Vaswani et al., NeurIPS 2017)

- Вся последовательность за один проход (параллельно).
- Self-Attention: каждый токен напрямую связан с любым другим за $O(1)$ шагов.
- Никаких RNN-слоёв: только Attention + FFN.
- Сложность $O(n^2)$, на практике $n \leq$ 512–1024.

2 Общая архитектура: Encoder–Decoder

Transformer состоит из $N = 6$ блоков энкодера и 6 блоков декодера.

2.1 Encoder

Каждый блок энкодера:

1. Multi-Head Self-Attention.
2. Add & Layer Normalization (residual connection).
3. Feed-Forward Network (FFN).
4. Add & Layer Normalization.

2.2 Decoder

Каждый блок декодера:

1. Masked Multi-Head Self-Attention (каузальная маска, запрет на будущие токены).
2. Add & Layer Normalization.
3. Cross-Attention (Encoder–Decoder Attention): Q из Decoder, K, V из Encoder.
4. Add & Layer Normalization.
5. Feed-Forward Network.
6. Add & Layer Normalization.

3 Входные представления

3.1 Токенизация и Embedding

Стандартный pipeline: токенизация (BPE, WordPiece, Unigram) $\to$ Embedding $V \times d_{\text{model}}$.

3.2 Positional Encoding

Так как Self-Attention инвариантен к порядку, необходимо добавить позиционную информацию:

$$\text{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Интуиция: каждая размерность — синусоида со своей частотой. Младшие $i \to$ высокая частота (точная позиция), старшие $i \to$ низкая частота (глобальный контекст). Сумма, а не конкатенация: $X = \text{TokenEmbedding}(X) + \text{PositionalEncoding}(X)$.

Современные альтернативы:

- Обучаемые Positional Embedding (BERT, GPT).
- RoPE (Rotary Position Embedding) — поворот Q и K на угол, зависящий от позиции.
- ALiBi (2022) — линейный штраф на attention score по расстоянию.

4 Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

где $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$.

Q_i — «что ищет i-й токен?», K_j — «что предлагает j-й токен?». QK^T — матрица близости $n \times m$, softmax даёт распределение по строкам.

Зачем делить на $\sqrt{d_k}$? Если q, k — независимые случайные векторы с компонентами $N(0,1)$, то $\mathrm{Var}(q \cdot k) = d_k$. Без деления softmax получает большие значения $\to$ острые распределения, градиент около нуля. Деление нормализует дисперсию к 1.

Интуиция: Attention = информационный поиск. Q = поисковый запрос, K = аннотация документов, V = содержание документов. Self-Attention: Q, K, V — всё из одной последовательности. Каждый токен «смотрит» на все остальные и собирает релевантную информацию (контекстуализация).

5 Multi-Head Attention

Одна голова — один тип отношений. Multi-Head = h параллельных «экспертов»:

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) W_O$$

$$\mathrm{head}_i = \mathrm{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

$h = 8$, $d_k = d_v = d_{\mathrm{model}}/h = 64$. Параметры на один слой attention: ≈ 1.05M. $\sim 70\%$ параметров блока — в FFN ($d_{ff} = 2048$).

6 Feed-Forward Network (FFN)

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

Point-wise — один FFN к каждому токену. $d_{ff} = 4 \cdot d_{\mathrm{model}} = 2048$. FFN — «память» модели: запоминает комбинации признаков. Современные варианты: GeLU (BERT, GPT), SwiGLU/GLU variants (Llama).

7 Residual Connections и Layer Normalization

$$x_{l+1} = \mathrm{LayerNorm}(x_l + \mathrm{Sublayer}(x_l))$$

Градиент проходит напрямую через residual connection:

$$\frac{\partial L}{\partial x_l} = \frac{\partial L}{\partial x_{l+1}} \left(1 + \frac{\partial \mathrm{Sub}}{\partial x_l}\right)$$

LayerNorm: нормализация по признакам (не по батчу):

$$\mathrm{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma} + \beta$$

8 Маскирование

Padding Mask: $-\infty$ для padding-токенов — softmax даёт 0, токен игнорируется.

Look-Ahead Mask (Decoder): верхняя треугольная матрица с $-\infty$ — токен на i видит только $\leq i$. Предотвращает подглядывание в будущее.

9 Обучение

Loss: Cross-Entropy, каждый шаг декодирования — свой loss.

Teacher Forcing: на каждом шаге подаются реальные токены (ускоряет обучение, но даёт exposure bias).

Optimizer: Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 10^{-9}$) с rate schedule: linear warmup (4k шагов) + inverse sqrt decay.

10 Инференс

Авторегрессивная генерация: на каждом шаге подаём сгенерированную последовательность. **KV-кэширование** — $K_{\text{enc}}, V_{\text{enc}}$ считаются один раз, на каждом шаге декодера — только для нового токена.

Стратегии декодирования: greedy (argmax), beam search ($B = 4$–8), top-K sampling, nucleus (top-p) sampling. Temperature: $\tau \in [0.7, 1.5]$.

11 Применения

- **BERT (Encoder-only)**: маскированный LM + NSP, 110M–340M параметров. Классификация, NER, QA.
- **GPT (Decoder-only)**: авторегрессивный LM. GPT-3: 175B, GPT-4: $\sim$ 1.8T. Few-shot, in-context learning.
- **T5**: все задачи $\to$ текст (text-to-text).
- Другие: BART, RoBERTa, Llama, Mistral, Gemini.
- Мультимодальные: ViT, CLIP, Whisper, Flamingo.

12 Выводы

Преимущества: параллелизация, долгосрочные зависимости за $O(1)$ шагов, скейлинг до 1.8T+ параметров, универсальность.

Ограничения: $O(n^2)$ сложность (дорого при $n > 4096$), нет врождённой позиции, требует много данных, дорогой инференс.

Решение для длинных последовательностей: Longformer, Reformer, Linformer, Mamba (SSM).